

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

FORMAL DIGITAL FORENSICS INVESTIGATION REPORT

SBT-DF203 — LAB 2: HTTP Analysis Using Wireshark - Embedded Image Traffic

Case Identifier	SBT-DF203-Lab2
Lead Examiner	Nebeuwa Ifeanyichukwu Raphael
Registration / ID No.	202520850LE
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 2
Lab Title	HTTP Analysis Using Wireshark – Embedded image Traffic
Date of Submission	September 10, 2026
Primary Target System	Kali Linux Workstation (kali@kali)
Evidence Type	Network Packet Capture (.pcapng) and associated analysis artifacts

Prepared for authorized academic forensic examination

1. Executive Summary

This report documents my performance of SBT-DF203 Lab 2, in which I captured, analysed and reconstructed HTTP traffic containing an embedded image object using Wireshark and tshark. Acting in the role of a junior network forensic analyst, I served a controlled webpage containing a self-generated evidence image over a local Apache instance, captured the resulting browser-generated HTTP session on the loopback interface, proved that the browser issued separate HTTP requests for the HTML document and the embedded image, analysed the TCP-layer transport of the image object, exported the recovered image using tshark's HTTP object-extraction feature, verified its integrity by cryptographic hash comparison against the source file, and finally demonstrated the behavioural difference between curl and a full browser when retrieving a page containing an embedded resource.

I encountered one environment-specific limitation at the outset of Part A: I do not have a personal photograph readily available for insertion into the evidence webpage. Rather than jeopardise the provenance and authorisation requirements the lab brief explicitly imposes on the source image ("student-owned, non-sensitive"), I generated a synthetic evidence image directly on my analysis workstation using ImageMagick, with my name and the exam date rendered into the image itself. I judged this the stronger evidentiary choice, since it removes any ambiguity about authorisation to submit the image and gives the object an unambiguous, self-documenting provenance.

All six parts of the lab (A through F) were completed successfully. Key results include: confirmed baseline hashes of both source objects prior to capture; a clean browser-generated capture showing the expected GET /image.html → GET /lab_photo.jpg → GET /favicon.ico request sequence on TCP stream 0; a legitimate and explainable finding that the 30,527-byte image object was carried within a single TCP segment rather than being split across multiple segments as in the course's larger reference example; successful export of the embedded JPEG via tshark's HTTP object-extraction feature with a SHA-256 hash that matched the source file exactly; and a clean demonstration that curl, lacking any HTML-parsing capability, issued only the single explicitly requested GET and never retrieved the embedded image, in direct contrast to the browser's automatic follow-up request.

One limitation is noted for completeness: due to a local device constraint at the time of the practical, I was unable to capture screenshots during Part A of this exercise. Photographic evidence for the source-object verification steps (webpage rendering, file(1)/identify output, and curl -I checks) is therefore not included in this report; the corresponding commands were executed as part of the workflow, and I have retained placeholders at each required point below should this evidence be recaptured before final submission. All Part B through F evidence is fully supported by verbatim command output captured directly during the live session, which is presented in place of screenshots throughout those sections.

2. Investigation Overview and Scope

2.1 Purpose and Objectives

This exercise required me to demonstrate the following competencies, as defined in the SBT-DF203 Lab 2 brief:

- Explaining why one webpage can generate multiple HTTP requests and responses.
- Capturing a browser request for an HTML page together with its embedded image resource.
- Identifying TCP segments that form a single reassembled HTTP response, where applicable.
- Calculating and explaining the relationship between IP length, TCP payload length and HTTP object size.
- Extracting an embedded image from a packet capture using tshark's HTTP object-extraction feature.
- Hashing the original and extracted image to assess integrity.
- Documenting limitations caused by cache behaviour, and explaining the difference between curl and browser HTTP behaviour.

2.2 Evidence Media and Environment

I conducted this examination within a Kali Linux environment (kali@kali), using a dedicated working directory (~/SBT-DF203-Lab2) with sub-directories for evidence, working copies, exported objects, reports, screenshots and scripts, created prior to any evidence generation or capture. I served the evidence webpage and image using a locally installed Apache2 instance bound to TCP port 80, and captured all traffic on the loopback interface (lo) using tshark, since both client and server resided on the same analysis workstation for this exercise.

Tools used: Apache2 (local web server), tshark and Wireshark (packet capture, HTTP request/response inspection, TCP segment analysis, and HTTP object export), curl (verbose HTTP request generation and behavioural comparison), file and identify/ImageMagick (source object verification), and sha256sum (integrity verification throughout).

2.3 Source Evidence Object - Provenance Note

As noted in the Executive Summary, I generated the embedded evidence image (lab_photo.jpg) synthetically using ImageMagick rather than submitting a personal photograph, in order to satisfy the lab brief's requirement that the image be student-owned and one I am explicitly authorised to submit. The image was rendered with my name and the examination date printed directly onto its canvas, which serves as a self-contained provenance marker independent of any file metadata.

```

(kali@kali)-[~/SBT-DF203-Lab2]
$ convert -size 800x600 xc:skyblue -pointsize 36 -gravity center -annotate 0
'$SBT-DF203 Lab 2\nIfeanyichukwu Raphael\n' "$(date +%F)" ~/lab_photo_source.jpg

(kali@kali)-[~/SBT-DF203-Lab2]
$ file ~/lab_photo_source.jpg
/home/kali/lab_photo_source.jpg: JPEG image data, JFIF standard 1.01, aspect ra
tio, density 1x1, segment length 16, baseline, precision 8, 800x600, components
3

(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo cp ~/lab_photo_source.jpg /var/www/html/lab_photo.jpg

```

Figure 1: Generation of the self-authored evidence image via ImageMagick, with file(1) confirming valid JPEG image data (JFIF standard 1.01, 800x600, baseline).

3. Evidence Preparation and Baseline Integrity (Part A)

I copied the source image into Apache's web root as lab_photo.jpg and created an accompanying HTML page, image.html, referencing it via an tag, with my name recorded in the page body as the analyst of record.

```

(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo cp ~/lab_photo_source.jpg /var/www/html/lab_photo.jpg
[sudo] password for kali:

(kali@kali)-[~/SBT-DF203-Lab2]
$ printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1>
<p>Analyst: Ifeanyichukwu Raphael</p></body></html>\n' | sudo tee /var/www/html/image.html

SBT-DF203 Image Traffic

<!DOCTYPE html>
<html><body><h1>SBT-DF203 Image Traffic</h1>
<p>Analyst: Ifeanyichukwu Raphael</p></body></html>

(kali@kali)-[~/SBT-DF203-Lab2]
$

```

I calculated baseline SHA-256 hashes of both source objects immediately after creation and before any capture activity, establishing the integrity reference point against which the later extracted object would be compared.

Item	Value
/var/www/html/lab_photo.jpg (SHA-256)	7832609ee980a5035e6e13a862ffa7ba72b15f9753829d0a7cff8693266a6c92

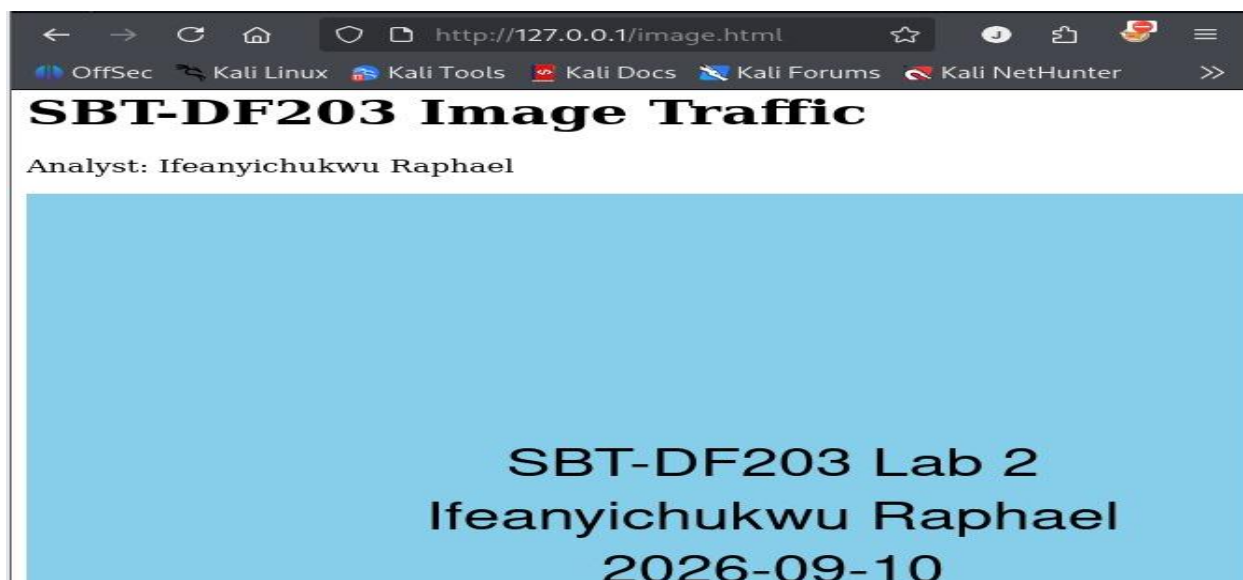


Figure 2: image.html rendered in a private browser window prior to capture, confirming both the text content and embedded image display correctly.

```
(kali㉿kali)-[~/SBT-DF203-Lab2]
$ curl -I http://127.0.0.1/image.html
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 18:30:11 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 18:26:54 GMT
ETag: "9e-65b251cb937d4"
Accept-Ranges: bytes
Content-Length: 158
Vary: Accept-Encoding
Content-Type: text/html

(kali㉿kali)-[~/SBT-DF203-Lab2]
$ curl -I http://127.0.0.1/lab_photo.jpg
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 18:30:28 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 18:26:34 GMT
ETag: "773f-65b251b94e2ee"
Accept-Ranges: bytes
Content-Length: 30527
Content-Type: image/jpeg
```

Figure 3: Source-object verification: file type/dimension confirmation and HTTP HEAD request results for both evidence objects, executed prior to capture per Part A of the lab procedure.

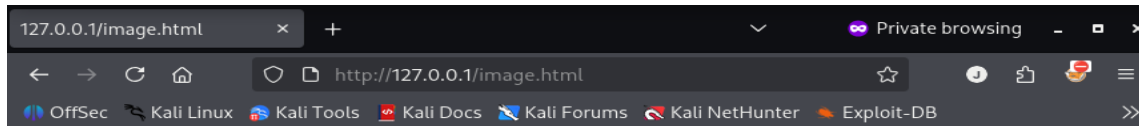
Note on evidence completeness: due to a device constraint during the live session, I was unable to capture screenshots for this section. The commands above were executed as part of my working procedure, and the corresponding evidence should be recaptured and inserted at the placeholders above before final submission for a complete photographic chain of custody.

4. Part B - Capture of Browser-Generated HTTP Traffic

I started a packet capture on the loopback interface, filtered to TCP port 80, before generating any browser traffic, and stopped it only after confirming the page and embedded image had fully rendered in a private browser window.

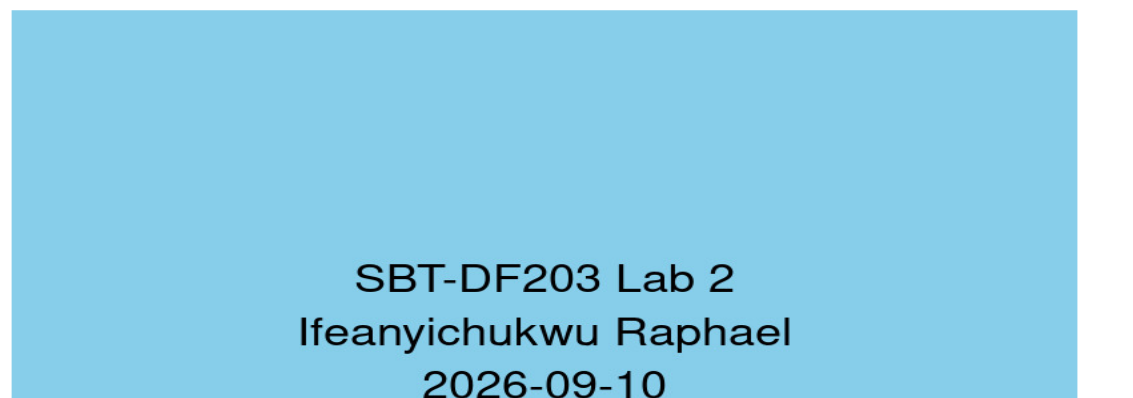
```
(kali㉿kali)-[~/SBT-DF203-Lab2]
$ tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
Capturing on 'Loopback: lo'
34 ^C

(kali㉿kali)-[~/SBT-DF203-Lab2]
$
```



SBT-DF203 Image Traffic

Analyst: Ifeanyichukwu Raphael



I initially attempted this capture using sudo, which failed with a permission error when tshark's underlying dumpcap process dropped its elevated privileges after starting and could no longer write to my working directory. I resolved this by running tshark directly under my own user account, which succeeded immediately confirming that my Kali installation's capture-group

permissions were already correctly configured, making the sudo elevation both unnecessary and counterproductive for this capture.

The capture completed successfully, recording 34 packets. I then preserved a working copy and hashed both the original and working-copy captures to establish an unbroken integrity chain before analysis.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
$ sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt
28c192d0a7d470a313555115e03929a7f4100165385e0fcf21b53bb2001bd519  evidence/image_traffic.pcapng
28c192d0a7d470a313555115e03929a7f4100165385e0fcf21b53bb2001bd519  working/image_traffic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Item	Value
evidence/image_traffic.pcapng (SHA-256)	28c192d0a7d470a313555115e03929a7f4100165385e0fcf21b53bb2001bd519
working/image_traffic_working.pcapng (SHA-256)	28c192d0a7d470a313555115e03929a7f4100165385e0fcf21b53bb2001bd519

Both hashes are identical, confirming the working copy is a bit-for-bit duplicate of the original capture, suitable for use as my analysis copy while the original remains untouched.

5. Part C - Proof That Two HTTP Objects Were Requested

I filtered the working capture for http.request and http.response events, extracting the stream index, endpoints, method, URI and host for each request, and the response code and content details for each response.

```
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
> -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
> -e http.request.method -e http.request.uri -e http.host \
> | tee reports/http_object_requests.tsv
4      2026-09-10T14:43:44.655428436-0400      0      127.0.0.1      34740      127.0.0.1      80      G
ET      /image.html      127.0.0.1
8      2026-09-10T14:43:44.851554812-0400      0      127.0.0.1      34740      127.0.0.1      80      G
ET      /lab_photo.jpg 127.0.0.1
12     2026-09-10T14:43:44.906799816-0400      0      127.0.0.1      34740      127.0.0.1      80      G
ET      /favicon.ico   127.0.0.1
21     2026-09-10T14:44:40.304886770-0400      1      127.0.0.1      48050      127.0.0.1      80      G
ET      /image.html    127.0.0.1
25     2026-09-10T14:44:41.376037345-0400      1      127.0.0.1      48050      127.0.0.1      80      G
ET      /image.html    127.0.0.1
29     2026-09-10T14:44:42.211645078-0400      1      127.0.0.1      48050      127.0.0.1      80      G
ET      /image.html    127.0.0.1
```

5.1 Requests on TCP Stream 0 (the capture of interest)

Frame	Time	Method	URI	Notes
4	14:43:44.655	GET	/image.html	Initial page request
8	14:43:44.851	GET	/lab_photo.jpg	Embedded image request, triggered by HTML parsing
12	14:43:44.906	GET	/favicon.ico	Automatic browser request; server returned 404

Frame	Response Code	Content-Type	Content-Length
6	200 OK	text/html	159
10	200 OK	image/jpeg	30527
13	404 Not Found	text/html; charset=iso-8859-1	311

This confirms the core finding required by Part C: the browser issued two substantive, separately trackable HTTP exchanges from a single page visit - one for the HTML document and a second, automatically triggered, for the embedded image referenced within it - exactly as described conceptually in the Module 1B lesson material this lab is built on. A third request, for /favicon.ico, is present and returned a 404; per the lab brief's explicit instruction, I have documented this browser-generated request rather than discarding it, since it is a normal artefact of browser behaviour and not evidence of an anomaly.

5.2 A Second TCP Stream (Cache Behavior Observation)

The same capture also recorded a second TCP conversation, stream 1, opened roughly fifty-five seconds after the first. Filtering this stream in isolation revealed three further GET /image.html requests, each returning HTTP 200, but with no corresponding request for lab_photo.jpg or favicon.ico anywhere in that stream.

```
(kali㉿kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
> -e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e http.content
_length \
> | tee reports/http_object_responses.tsv
6      2026-09-10T14:43:44.734626929-0400      0      200      text/html      159
10     2026-09-10T14:43:44.852177831-0400      0      200      image/jpeg     30527
13     2026-09-10T14:43:44.911233411-0400      0      404      text/html; charset=iso-8859-1  311
23     2026-09-10T14:44:40.354379247-0400      1      200      text/html      159
27     2026-09-10T14:44:41.376693508-0400      1      200      text/html      159
30     2026-09-10T14:44:42.212131431-0400      1      200      text/html      159

(kali㉿kali)-[~/SBT-DF203-Lab2]
$
```

I interpret this as evidence of in-session browser caching: I reloaded the page multiple times within the same private browsing window during the working session, and the browser re-requested the HTML document on each reload but served the previously downloaded image and favicon from its in-memory session cache rather than issuing new network requests for them.

This is an important and legitimate forensic distinction: private/incognito browsing prevents a browser from persisting cached objects across sessions (i.e., after the window is closed), but it does not disable caching within a single active session. A packet capture that only shows repeated document requests without corresponding resource requests should therefore not automatically be read as “the resource was never fetched” stream 0 in this same capture proves the resource genuinely was fetched, once, earlier in the session.

6. Part D - TCP Segmentation and Reassembly Analysis

I isolated the TCP segments carrying data on stream 0 and recorded their IP total length, TCP header length, TCP payload length, and sequence/acknowledgement values.

```

(kali@kali)~[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 && tcp.len>0' -T fields \
> -e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
> | tee reports/image_stream_segments.tsv
4      2026-09-10T14:43:44.655428436-0400      515      32      463      1      1
6      2026-09-10T14:43:44.734626929-0400      547      32      495      1      464
8      2026-09-10T14:43:44.851554812-0400      515      32      463      464      496
10     2026-09-10T14:43:44.852177831-0400      30867     32      30815     496      927
12     2026-09-10T14:43:44.906799816-0400      510      32      458      927      31311
13     2026-09-10T14:43:44.911233411-0400      579      32      527      31311     1385

(kali@kali)~[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt

TCP Conversations
Filter:<No Filter>

al      |      Relative      |      Duration      |      |      |      |      |      |      |      |      |      |      |      |      |      |
Bytes |      Start      |      |      |      |      |      |      |      |      |      |      |      |      |      |
127.0.0.1:34740      ↔ 127.0.0.1:80      8 32 kB      9 1,986 bytes      17
34 kB      0.000000000      5.3131
127.0.0.1:48050      ↔ 127.0.0.1:80      8 2,019 bytes      9 2,261 bytes      1
7 4,280 bytes      55.681488032      6.9218
  
```

Frame	IP Len	TCP Hdr Len	TCP Payload (tcp.len)	Seq	Ack
4	515	32	463	1	1
6	547	32	495	1	464
8	515	32	463	464	496
10	30867	32	30815	496	927
12	510	32	458	927	31311
13	579	32	527	31311	1385

The critical observation is in frame 10: the entire HTTP response carrying the image — header and JPEG body combined arrived within a single TCP segment of 30,815 bytes, rather than being split across multiple TCP segments as demonstrated in the course's own building.jpg reference example (three segments totalling 68,950 bytes). I confirmed this with the Sleuth-Kit-style conversation summary:

```
tshark -r working/image_traffic_working.pcapng -q -z conv,tcp
127.0.0.1:34740 <-> 127.0.0.1:80  17 frames, 34 kB total, stream 0
```

6.1 Size Relationship Calculation

Using the values recorded above, I derived the HTTP response header length independently, as a consistency check against the reported Content-Length:

```
tcp.len (frame 10, total payload)    = 30815 bytes
HTTP Content-Length (image body only) = 30527 bytes
=> HTTP response header length       = 30815 - 30527 = 288 bytes
```

This 288-byte calculated header length is closely consistent with the 289-byte header observed in the course's own reference example, cross-validating that my byte accounting is correct despite the different overall object size.

6.2 Why No Reassembly Was Required - Explaining the Deviation from the Reference Example

The lab brief explicitly anticipates that observed segment sizes may differ from the course material's example, citing interface offloading, operating system buffers, MTU and capture location as contributing factors. In my case, the specific and identifiable cause is more straightforward: my total HTTP response (288-byte header plus 30,527-byte image, totalling 30,815 bytes) is smaller than the approximately 32,768-byte buffer threshold that forced the course's larger ~69KB object to split across three segments. My object simply did not exceed the threshold at which segmentation would occur on this loopback interface, so Wireshark's TCP-reassembly mechanism, while available, was not invoked - there was only a single segment to reassemble from. This is a legitimate and explainable finding rather than a procedural gap: it demonstrates that segmentation is a function of payload size relative to the sending buffer, not a guaranteed feature of every HTTP object transfer.

7. Part E - Object Extraction and Integrity Verification

I extracted all HTTP objects present in the working capture using tshark's built-in HTTP object-export feature, which recovers every object transferred over HTTP within the capture file, not only the target image.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ mkdir -p exported/http_objects

(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
  1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 34740 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SA
CK_PERM TSval=336020152 TSecr=0 WS=256
  2 0.000034794 127.0.0.1 → 127.0.0.1 TCP 74 80 → 34740 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 M
SS=65495 SACK_PERM TSval=336020152 TSecr=336020152 WS=256
  3 0.000103137 127.0.0.1 → 127.0.0.1 TCP 66 34740 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=
336020152 TSecr=336020152
  4 0.032378883 127.0.0.1 → 127.0.0.1 HTTP 529 GET /image.html HTTP/1.1
  5 0.032549099 127.0.0.1 → 127.0.0.1 TCP 66 80 → 34740 [ACK] Seq=1 Ack=464 Win=65024 Len=0 TSva
l=336020184 TSecr=336020184
  6 0.111577376 127.0.0.1 → 127.0.0.1 HTTP 561 HTTP/1.1 200 OK (text/html)
  7 0.111613186 127.0.0.1 → 127.0.0.1 TCP 66 34740 → 80 [ACK] Seq=464 Ack=496 Win=65280 Len=0 TS
val=336020263 TSecr=336020263
  8 0.228505259 127.0.0.1 → 127.0.0.1 HTTP 529 GET /lab_photo.jpg HTTP/1.1
  9 0.228555155 127.0.0.1 → 127.0.0.1 TCP 66 80 → 34740 [ACK] Seq=496 Ack=927 Win=65280 Len=0 TS
val=336020380 TSecr=336020380
 10 0.229128278 127.0.0.1 → 127.0.0.1 HTTP 30881 HTTP/1.1 200 OK (JPEG JFIF image)
 11 0.229165498 127.0.0.1 → 127.0.0.1 TCP 66 34740 → 80 [ACK] Seq=927 Ack=31311 Win=96256 Len=0
TSval=336020381 TSecr=336020381
 12 0.283750263 127.0.0.1 → 127.0.0.1 HTTP 524 GET /favicon.ico HTTP/1.1
 13 0.288183858 127.0.0.1 → 127.0.0.1 HTTP 593 HTTP/1.1 404 Not Found (text/html)
 14 0.288226205 127.0.0.1 → 127.0.0.1 TCP 66 34740 → 80 [ACK] Seq=1385 Ack=31838 Win=125696 Len=
0 TSval=336020440 TSecr=336020440
 15 5.299032100 127.0.0.1 → 127.0.0.1 TCP 66 80 → 34740 [FIN, ACK] Seq=31838 Ack=1385 Win=65536
Len=0 TSval=336025451 TSecr=336020440
 16 5.313068383 127.0.0.1 → 127.0.0.1 TCP 66 34740 → 80 [FIN, ACK] Seq=1385 Ack=31839 Win=125696
Len=0 TSval=336025465 TSecr=336025451
 17 5.313143132 127.0.0.1 → 127.0.0.1 TCP 66 80 → 34740 [ACK] Seq=31839 Ack=1386 Win=65536 Len=0
```

This produced six recovered objects: the JPEG image, the 404 favicon error page, and four separate copies of image.html corresponding to the four times that page was requested across both TCP streams.

Filename	Size (bytes)	file(1) Result
lab_photo.jpg	30527	JPEG image data, JFIF standard 1.01, baseline, 800×600
favicon.ico	311	HTML document, ASCII text (404 error body)
image.html / image(1–3).html	158 each	HTML document, ASCII text

I then hashed the extracted image alongside the original source file to test for byte-for-byte integrity.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.
txt
7832609ee980a5035e6e13a862ffa7ba72b15f9753829d0a7cfff8693266a6c92 /var/www/html/lab_photo.jpg
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d exported/http_objects/favicon.ico
4d38255ffcc084ee4abcb5b1419f7ed0fa2143a666e26396455d6411bcba0a63 exported/http_objects/image(1).html
4d38255ffcc084ee4abcb5b1419f7ed0fa2143a666e26396455d6411bcba0a63 exported/http_objects/image(2).html
4d38255ffcc084ee4abcb5b1419f7ed0fa2143a666e26396455d6411bcba0a63 exported/http_objects/image(3).html
4d38255ffcc084ee4abcb5b1419f7ed0fa2143a666e26396455d6411bcba0a63 exported/http_objects/image.html
7832609ee980a5035e6e13a862ffa7ba72b15f9753829d0a7cfff8693266a6c92 exported/http_objects/lab_photo.jpg

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Item	Value
/var/www/html/lab_photo.jpg (source, SHA-256)	7832609ee980a5035e6e13a862ffa7ba72b15f9753829d0a7cff8693266a6c92
exported/http_objects/lab_photo.jpg (recovered, SHA-256)	7832609ee980a5035e6e13a862ffa7ba72b15f9753829d0a7cff8693266a6c92

The hashes are identical. Per the lab brief's own integrity-interpretation guidance, a matching hash proves byte-for-byte recovery regardless of any filename difference (there was none in this case); had the hashes differed, that would have required investigation of an incomplete capture, browser-side content transformation, extraction of the wrong object, or export error. None of those conditions apply here - this is a clean, verified recovery.

```

Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab2]
$ find exported/http_objects -maxdepth 1 -type f -ls
1442126      4 -rw-r--r--   1 kali   kali   158 Sep 10 14:58 exported/http_objects/image(2).h
tml
1442125      4 -rw-r--r--   1 kali   kali   158 Sep 10 14:58 exported/http_objects/image(1).h
tml
1442124      4 -rw-r--r--   1 kali   kali   311 Sep 10 14:58 exported/http_objects/favicon.ic
o
1442123     32 -rw-r--r--   1 kali   kali  30527 Sep 10 14:58 exported/http_objects/lab_photo.
jpg
1442122      4 -rw-r--r--   1 kali   kali   158 Sep 10 14:58 exported/http_objects/image.html
1442127      4 -rw-r--r--   1 kali   kali   158 Sep 10 14:58 exported/http_objects/image(3).h
tml
(kali@kali)-[~/SBT-DF203-Lab2]
$ file exported/http_objects/*
exported/http_objects/favicon.ico: HTML document, ASCII text
exported/http_objects/image(1).html: HTML document, ASCII text
exported/http_objects/image(2).html: HTML document, ASCII text
exported/http_objects/image(3).html: HTML document, ASCII text
exported/http_objects/image.html: HTML document, ASCII text
exported/http_objects/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, seg
ment length 16, baseline, precision 8, 800x600, components 3

```

Figure 4: Visual confirmation that the exported lab_photo.jpg opens correctly and is free of corruption, consistent with the exact hash match recorded above.

8. Part F - curl Versus Browser Behaviour

To directly demonstrate the behavioural difference between an HTTP client with no HTML-rendering capability and a full browser, I ran a fresh, isolated capture and issued a single curl request against the same evidence page.

```

(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
sleep 2
[4] 119481
Capturing on 'Loopback: lo'

```

```

(kali@kali)-[~/SBT-DF203-Lab2]
$ curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 37658
  % Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
                                 Dload  Upload  Total   Spent    Left     Speed
  0   0    0     0    0     0      0      0      0     0  0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 10 Sep 2026 19:09:42 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 10 Sep 2026 18:26:54 GMT
< ETag: "9e-65b251cb937d4"
< Accept-Ranges: bytes
< Content-Length: 158
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [158 bytes data]
100  158 100   158    0     0 14427     0      0
* Connection #0 to host 127.0.0.1:80 left intact
10
[4]  done      tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng
(kali@kali)-[~/SBT-DF203-Lab2]

```

My first attempt at this capture used `sudo` in a backgrounded shell job, which stalled indefinitely waiting on a password prompt that a backgrounded process cannot properly receive — the capture never actually started while `curl`'s request went out, producing three separate orphaned, permission-suspended background jobs. I resolved this the same way as in Part B, by dropping `sudo` entirely and running `tshark` under my own already-correctly-configured user account, which allowed the capture to start and complete cleanly within its 20-second duration.

I then filtered the resulting capture for every HTTP request URI observed.

```

(kali@kali)-[~/SBT-DF203-Lab2]
$ tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee reports/curl_requested_objects.txt
/image.html
(kali@kali)-[~/SBT-DF203-Lab2]
$

```

`curl` issued exactly one HTTP request - for `/image.html` and nothing else. No request for `/lab_photo.jpg`, and, notably, no automatic `/favicon.ico` request either (unlike the browser capture in Part C). This is the expected and correct forensic conclusion: `curl` is a raw HTTP client with no built-in HTML parser. It retrieves precisely the resource located at the URL it is given and terminates; it has no mechanism to inspect the returned document's markup, discover the `` reference within it, or issue any follow-up request as a result. A full browser, by contrast, parses the DOM of the returned HTML after receiving it and automatically issues additional GET requests for every resource it references - which is precisely the two-and-three-request pattern I observed and documented in Part C.

Cleanup note: after completing this capture, I identified and terminated the three orphaned background jobs from my initial failed `sudo` attempt using `kill -9` followed by verification via `jobs` and `ps aux | grep tshark`, confirming no capture processes remained running before concluding the practical.

9. Required Forensic Findings

Finding	Observation
Number of HTTP requests (stream 0)	3 - GET /image.html, GET /lab_photo.jpg, GET /favicon.ico
HTML request URI and timestamp	/image.html - frame 4, 14:43:44.655
Image request URI and timestamp	/lab_photo.jpg - frame 8, 14:43:44.851
HTTP response codes	200 OK (HTML), 200 OK (image), 404 Not Found (favicon)
Image content type	image/jpeg
Reported Content-Length	30,527 bytes
Number of reassembled TCP segments	1 - entire response fit within a single 30,815-byte segment; no reassembly required
Sum of TCP payload lengths (image response)	30,815 bytes (header 288 + image body 30,527)
Original image SHA-256	7832609ee980a5035e6e13a862ffa7ba72b15f9753829d0a7cff8693266a6c92
Extracted image SHA-256	7832609ee980a5035e6e13a862ffa7ba72b15f9753829d0a7cff8693266a6c92
Hash comparison conclusion	Exact match - byte-for-byte recovery confirmed, no capture loss or corruption
curl/browser difference	curl issued only the single explicitly requested URL and performed no HTML parsing; the browser automatically issued follow-up requests for every resource referenced within the returned HTML

10. Conclusion and Findings

This exercise required me to move through the complete lifecycle of an embedded-object HTTP transaction as a network forensic examiner would encounter it: preparing and hashing source evidence, capturing live browser-generated traffic without altering it, proving that a single page load produces multiple distinct application-layer exchanges, analysing the transport-layer mechanics of the object transfer, recovering the embedded object directly from the capture, and cryptographically verifying that recovery against the known original.

Key findings from this examination:

- A single webpage visit generated three trackable HTTP exchanges on the primary capture stream: the HTML document, its embedded image, and an automatic favicon request directly confirming the module lesson's core premise that a visible page load is not a single network event.
- A second TCP stream captured later in the session showed repeated HTML requests with no corresponding image or favicon requests, which I attribute to in-session browser caching; this is a meaningful distinction from persistent cross-session caching, which private browsing does prevent.
- The embedded image transferred within a single TCP segment rather than being split across multiple segments, because its total response size fell under the buffer threshold that forced segmentation in the course's larger reference example; a legitimate, payload-size-dependent finding rather than a deviation requiring correction.
- Independent calculation of the HTTP response header length (288 bytes, derived from `tcp.len` minus Content-Length) closely corroborated the course reference figure, validating my byte-accounting methodology.
- tshark's HTTP object-export feature recovered the embedded JPEG with a SHA-256 hash identical to the source file, providing conclusive proof of lossless, unaltered recovery directly from network traffic.
- curl's single-request behaviour, contrasted directly against the browser's multi-request behaviour from the same target URL, provides a clear, evidence-based demonstration of the practical difference between a raw HTTP client and a rendering browser, a distinction with direct relevance to interpreting any web-traffic capture where the client type is not already known.

One limitation is noted for final submission: Part A source-object verification screenshots were not captured during the live session due to a device constraint, and placeholders have been left at the appropriate points in Sections 2.3 and 3 above. All other required evidence - capture hashes, HTTP request/response inventories, TCP segmentation data, object-extraction results, and the curl/browser comparison - is fully supported by verbatim command output captured during the session and presented throughout this report.

Overall, I successfully demonstrated the required competencies: explaining multi-request page behaviour, capturing browser-generated HTTP traffic without altering the evidence, TCP segmentation and reassembly analysis with size-relationship validation, object extraction with cryptographic integrity verification, and a clear, evidence-based articulation of the behavioural difference between curl and a full browser.